1. Foundations and Review

1.1 Course Introduction and Expectations

1.1.1 Course Overview

CS3400 builds on the programming and object-oriented programming knowledge students already have.

The central question of the course is:

> **How should we organize data so that the operations we need can be performed effectively?**

A program does not simply manipulate individual variables. Real applications work with collections of data:

- students in a university
- employees in an organization
- products in a store
- messages in a communication system
- files in a directory
- nodes in a network
- transactions in a database

The way this data is organized has a major effect on the efficiency, clarity, and maintainability of a program.

For example, suppose we need to store 1,000,000 student numbers and frequently search for a particular student.

We could use:

Array
Linked List
Binary Search Tree
Hash Table

All four can store the same information, but they do not provide the same performance for searching.

Therefore:

> **Choosing an appropriate data structure is an important part of algorithm and software design.**

1.1.2 What Students Should Be Able to Do

By the end of CS3400, students should be able to:

1. Explain common data structures and their characteristics.
2. Implement data structures in C++.
3. Understand the operations supported by different structures.
4. Analyse the time and space requirements of operations.
5. Select an appropriate data structure for a particular problem.
6. Understand the relationship between ADTs and their implementations.
7. Implement structures using dynamic memory and pointers.
8. Use templates to develop reusable data structures.
9. Understand how different implementations affect performance.

The emphasis should not be:

> **"Can you memorize the definition of a linked list?"**

but rather:

> **"Can you decide when a linked list is appropriate, implement it correctly, and explain the cost of its operations?"**

1.2 Mathematical Background for Algorithms

This section is primarily a **recap**.

Students have already encountered mathematical concepts through Discrete Mathematics. The goal here is to establish the mathematical language that will be used throughout CS3400.

1.2.1 Mathematical Notation

Computer science frequently uses mathematical notation to describe algorithms and data.

For example:

n

usually represents the **input size**.

If an algorithm processes an array containing 100 elements:

$n=100$

If it processes an array containing 10,000 elements:

$n=10000$

When analysing algorithms, we are generally interested in how the amount of work changes as n increases.

1.2.2 Functions and Growth

A function describes a relationship between an input and an output.

For example:

$f(n)=2n+5$

If $n=10$:

$f(10)=25$

In algorithm analysis, functions are particularly important because we can use them to describe the amount of work performed by an algorithm.

For example:

```
for (int i = 0; i < n; i++)
{
    cout << i << endl;
}
```

The loop executes approximately n times.

Therefore, its running time can be described by:

$T(n)=an+b$

for some constants a and b.

We are usually interested in the **growth** of the function rather than the exact constants.

This leads to asymptotic analysis later in the course.

1.2.3 Common Growth Rates

Students should be comfortable recognizing the following:

1 $\log n$ n $n\log n$ n^2 n^3 2^n $n!$

The general relationship is:

$1 < \log n < n < n\log n < n^2 < n^3 < 2^n < n!$

for sufficiently large n.

This becomes important when comparing algorithms and data-structure operations.

For example, if one implementation performs:

n

operations while another performs:

n^2

operations, the difference becomes enormous as n grows.

1.2.4 Why This Matters in CS3400

Mathematics provides the language used to answer questions such as:

- How many operations does insertion require?
- How many comparisons are needed to search?
- How much memory does a structure require?
- What happens when the number of elements doubles?
- Which implementation scales better?

For example:

Operation	**Possible cost**
Access array element	O(1)
Search unsorted list	O(n)
Binary search	O(logn)
Nested loop over n elements	$O(n^2)$

The mathematical background therefore supports the analysis of the actual data structures we implement.

1.3 Sets, Relations, and Functions

This is another **targeted revision section**.

Students should already know the mathematical definitions. In CS3400, we are interested primarily in how these concepts help us describe data and computational relationships.

1.3.1 Sets

A **set** is a collection of distinct objects.

For example:

$S=\{2,4,6,8,10\}$

The objects in a set are called **elements**.

We write:

$4 \in S$

to mean that 4 belongs to S.

We write:

$5 \notin S$

to mean that 5 does not belong to S.

1.3.2 Common Set Operations

Union

The union of two sets contains elements belonging to either set.

$A \cup B$

Example:

$A=\{1,2,3\}$ $B=\{3,4,5\}$

Therefore:

$A \cup B=\{1,2,3,4,5\}$

Intersection

The intersection contains elements common to both sets.

$A \cap B$

Therefore:

$A \cap B=\{3\}$

Difference

$A-B$

contains elements in A that are not in B.

Therefore:

$A - B = \{1, 2\}$

1.3.3 Why Sets Matter in CS3400

Sets provide a useful mathematical model for collections of data.

Consider:

Student numbers
{1001, 1002, 1003, 1004}

We may be interested in operations such as:

- adding an element
- removing an element
- determining whether an element exists
- finding common elements
- finding elements that occur in one collection but not another

These ideas appear in many data structures.

For example, a **set ADT** might support:

insert(x)
remove(x)
contains(x)
size()
isEmpty()

Different data structures can implement the same conceptual set.

For example:

Set ADT
  |
  +---- Binary Search Tree
  |
  +---- Hash Table
  |
  +---- Sorted Array

This is an important CS3400 idea:

> **The mathematical concept describes what the collection represents; the data structure determines how it is implemented.**

1.3.4 Relations

A relation describes a relationship between elements.

For example, suppose:

$A=\{1,2,3\}$

A relation R might represent:

$R=\{(1,1),(2,2),(3,3)\}$

where each pair represents a relationship between two elements.

Relations can have properties such as:

Reflexive

Every element is related to itself.

aRa

Symmetric

If a is related to b, then b is related to a.

$aRb \Rightarrow bRa$

Transitive

If a is related to b, and b is related to c, then a is related to c.

$aRb \land bRc \Rightarrow aRc$

Students do not need an extensive treatment of relation theory here. The important point is understanding that **relationships between objects can be represented mathematically and computationally**.

1.3.5 Relations in Data Structures

Relations appear naturally in data structures.

Consider a university system:

Student ---- enrolled in ----> Course

Or a social network:

Alice ---- friends with ----> Bob

Or a road network:

Maseru ---- road ----> Roma

These relationships can eventually be represented using structures such as **graphs**.

Therefore:

> **Relations provide a mathematical foundation for structures that represent connections between objects.**

This becomes particularly important when we study graphs later in CS3400.

1.3.6 Functions

A function maps elements from one set to another.

We can write:

$f:A \rightarrow B$

This means that f maps elements of A to elements of B.

For example:

$f(x)=2x$

If:

$A=\{1,2,3\}$

then:

$f(A)=\{2,4,6\}$

Functions in Computer Science

Functions are extremely important in CS.

A C++ function:

```
int square(int x)
{
   return x * x;
}
```

can be viewed mathematically as:

$f(x)=x2$

Functions also appear when describing algorithms.

For example, we might describe the running time of an algorithm as:

$T(n)=n^2$

where:

- n = input size
- T(n) = amount of work performed

Functions therefore provide the mathematical language for describing algorithmic behaviour.

1.3.7 Hash Functions

One particularly important connection to CS3400 is the **hash function**.

A hash function maps a key to a position in a hash table:

$h:K\rightarrow\{0,1,\ldots,m-1\}$

For example:

$h(k) = k \bmod 10$

If:

$k=123$

then:

$h(123)=3$

The function determines where the data should be stored.

This provides a direct example of how mathematical functions become practical components of data structures. We will study hash table as time progresses

---

1.4 Summations, Recurrences, and Proof Techniques

Unlike Sets and Relations, this section has a **more direct connection to algorithm analysis**.

1.4.1 Summation Notation

A summation represents the addition of a sequence of terms.

$$\sum_{i=1}^{n} i$$

means:

1 + 2 + 3 + ··· + n

A common formula is:

$$\sum_{i=1}^{n} 2n(n+1)$$

Therefore:

1 + 2 + 3 + 4 + 5 = 15

because:

25(6)=15

Why Summations Matter

Summations frequently appear when counting operations.

Consider:

```
for (int i = 1; i <= n; i++)
{
   for (int j = 1; j <= i; j++)
   {
      // operation
   }
}
```

The inner loop executes:

1+2+3+···+n

times.

Therefore:

$$T(n) = \sum_{i=1}^{n} i$$

and:

T(n)=2n(n+1)

which is:

T(n)=O(n2)

Students therefore need summations not because CS3400 is a mathematics course, but because **summations help us count algorithmic work**.

1.4.2 Recurrence Relations

A recurrence describes a problem in terms of smaller instances of itself.

For example:

$T(n)=T(n-1)+1$

This says that solving the problem for n elements requires solving the problem for n−1 elements plus one additional unit of work.

This commonly occurs in recursive algorithms.

For example:

```
void print(int n)
{
   if (n == 0)
      return;

   cout << n << endl;
   print(n - 1);
}
```

The running time can be expressed as:

$T(n)=T(n-1)+c$

which gives:

$T(n)=O(n)$

Recurrences in CS3400

Recurrences become particularly useful when studying recursive data structures and algorithms.

Examples include:

- tree traversal
- recursive tree operations
- divide-and-conquer algorithms
- binary search

For example, binary search approximately halves the problem at every step:

$T(n)=T(n/2)+c$

which results in:

$T(n)=O(\log n)$

1.4.3 Mathematical Induction

Mathematical induction is a technique for proving that a statement is true for all relevant values of n.

It generally involves:

Step 1: Base Case

Show that the statement is true for the smallest value.

Step 2: Inductive Hypothesis

Assume that the statement is true for some $n=k$.

Step 3: Inductive Step

Show that if it is true for k, then it is also true for $k+1$.

Why Induction Matters in CS3400

Induction is useful for proving properties of:

- recursive algorithms
- trees
- linked structures
- algorithm correctness
- mathematical formulas

For example, when reasoning about the number of nodes in a binary tree or the correctness of a recursive traversal, induction provides a systematic way to construct a proof.

Students should understand the technique rather than spend excessive time on formal proof theory.

1.4.4 Basic Proof Techniques

Students should be familiar with:

- direct proof
- proof by contradiction
- proof by induction
- counterexamples

In CS3400, these techniques support reasoning about **correctness and properties of data structures and algorithms**.

For example:

> "A binary search tree always has its smaller keys in the left subtree."

A student should be able to reason about why this property must remain true after insertion.

1.5 C++ Recap: Data Structures and Abstraction

This section is important because CS3400 will use C++ extensively.

The students have already completed OOP, so this should not become a full OOP revision. Instead, revisit only the concepts that are needed to implement data structures properly.

1.5.1 Classes and Objects

A class defines the structure and behaviour of objects.

```
class Student
{
private:
    int studentNumber;
```

```
public:
   void setStudentNumber(int number)
   {
      studentNumber = number;
   }

   int getStudentNumber()
   {
      return studentNumber;
   }
};
```

A class can contain:

- data members
- member functions
- constructors
- destructors
- access control

Data structures will commonly be implemented as classes.

For example:

```
class Stack
{
   // implementation
};
```

1.5.2 Encapsulation and Data Hiding

Encapsulation combines data and operations that work on that data.

Data hiding restricts direct access to implementation details.

For example:

```
class Stack
{
private:
   int data[100];
   int top;
```

```
public:
   void push(int value);
   void pop();
   int peek();
};
```

Users of the class do not need to know how `push()` is implemented.

They only need to know:

```
push(value)
pop()
peek()
```

This idea provides the bridge from **OOP to Abstract Data Types**.

1.5.3 Dynamic Memory Management

Many data structures cannot assume a fixed number of elements.

Consider a linked list.

Every node may be dynamically created:

```
Node* newNode = new Node;
```

and eventually released:

```
delete newNode;
```

Failure to manage dynamically allocated memory correctly can result in **memory leaks**.

For example:

```
void createNode()
{
   Node* p = new Node;
}
```

When `createNode()` finishes, the pointer p disappears, but the dynamically allocated object remains.

The memory has been leaked.

1.5.4 RAII

C++ provides an important principle known as:

> **Resource Acquisition Is Initialization (RAII).**

The basic idea is:

> A resource is acquired by an object and automatically released when that object is destroyed.

Resources include:

- dynamically allocated memory
- files
- locks
- sockets
- database connections

For example:

```
{
   std::vector<int> numbers;
   // use numbers
}
// numbers is destroyed here
```

When the object leaves its scope, its destructor runs and releases the resources it owns.

This becomes particularly important when implementing data structures safely.

1.5.5 Pointers and References

Pointers store memory addresses.

```
int x = 10;
int* p = &x;
```

Here:

x = 10
p -----> x

Pointers are essential for structures such as:

- linked lists
- trees
- graphs

For example:

```
struct Node
{
   int data;
   Node* next;
};
```

The `next` pointer connects one node to another.

1.5.6 Shallow Copy and Deep Copy

This is particularly important when classes manage dynamic memory.

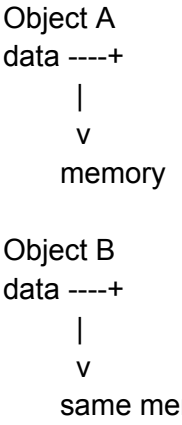
Consider:

```
class Array
{
private:
   int* data;

public:
   Array(int size)
   {
      data = new int[size];
   }
};
```

A **shallow copy** copies the pointer itself.

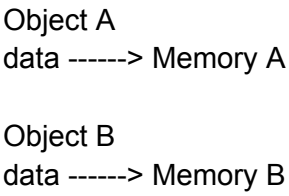
Therefore:



Both objects refer to the same dynamically allocated memory.

This can cause serious problems, particularly when one object is destroyed.

A **deep copy** creates a new allocation and copies the actual data:



The two objects own independent memory.

This leads directly to the **Rule of Three/Five**, constructors, destructors, and copy/move semantics when implementing resource-owning data structures.

1.5.7 Templates and Generic Programming

A data structure should ideally not be limited to one data type.

Instead of:

```
class Stack
{
   int data[100];
};
```

we can use a template:

```
template <typename T>
```

```
class Stack
{
private:
   T data[100];

public:
   void push(T value);
};
```

Now the same structure can store:

```
Stack<int>
Stack<double>
Stack<string>
```

This is an important concept because the data structures developed in CS3400 should be **reusable and generic**.

1.5.8 Abstraction and Interfaces

Abstraction means focusing on **what an object does** rather than **how it does it**.

Suppose we have a Stack.

The user needs to know:

```
push()
pop()
peek()
isEmpty()
```

The user does not necessarily need to know whether the stack is implemented using:

Array

or:

Linked List

This is the central idea behind an ADT.

1.5.9 C++ Classes and ADTs

An important distinction:

> **A class is a C++ language construct. An ADT is a conceptual specification of behaviour.**

For example:

Stack ADT

The Stack specifies operations such as:

push(x)
pop()
top()
isEmpty()

We can implement the Stack ADT using:

Array
Linked List

The ADT stays the same even though the implementation changes.

This distinction will be fundamental throughout CS3400.

---

2. Abstract Data Types and Algorithm Analysis

2.1 Abstract Data Types and Containers

2.1.1 What Is an ADT?

An **Abstract Data Type (ADT)** describes:

1. What data is represented.
2. What operations are available.
3. What those operations mean.

It does **not** specify the implementation.

For example, a Stack ADT follows the **Last-In, First-Out (LIFO)** principle.

Operations may include:

push(x)
pop()
top()
isEmpty()

The specification does not say whether the Stack must use an array or linked list.

2.1.2 ADT versus Data Structure

This distinction is extremely important.

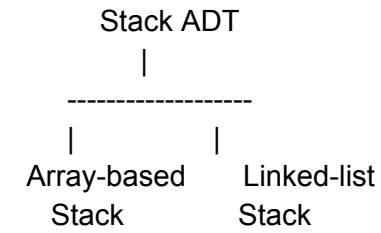
ADT

Describes **behaviour**.

Data Structure

Describes **representation and implementation**.

For example:



Both implementations provide the same conceptual Stack behaviour.

2.1.3 Containers

A **container** is a programming construct used to store and manage collections of objects.

Examples from the C++ Standard Library include:

std::vector
std::list
std::deque
std::stack
std::queue
std::set
std::map

Containers provide already-developed implementations of common data structures and collection abstractions.

For example:

std::vector<int> numbers;

provides a dynamic array-like container.

Students should understand that containers are not simply "ADTs". A container is generally a concrete programming component that provides a particular collection interface and implementation.

2.1.4 Relationship Between the Concepts

A useful way to visualize the concepts is:

Problem
  |
  v
ADT
"What behaviour do we need?"
  |
  v
Data Structure
"How should the data be organized?"
  |
  v
Implementation
"How do we implement it in C++?"
  |
  v
Algorithm
"How do we perform operations on it?"

For example:

Requirement:
Frequently insert and search student records

↓

Possible ADT:
Dictionary / Map

↓

Possible data structure:
Hash Table

↓

C++ implementation:
HashTable<StudentNumber, Student>

↓

Algorithms:
hashing, insertion, lookup, collision handling

2.2 Data Structures and Algorithms

A **data structure** is a systematic way of organizing and storing data so that it can be used effectively.

Examples:

- arrays
- linked lists
- stacks
- queues
- trees
- heaps
- hash tables
- graphs

An **algorithm** is a finite sequence of steps for solving a problem or performing a computation.

For example:

Data Structure: Binary Search Tree

Operations:
insert
search
delete
traverse

Each operation requires an algorithm.

Therefore:

> **Data structures and algorithms are closely related.**

A data structure determines what operations are convenient, while algorithms determine how those operations are performed.

2.2.1 Choosing a Data Structure

Consider the requirement:

> We need to repeatedly access elements using their position.

An array may be appropriate because:

A[i]

can normally be accessed in:

O(1)

Now consider:

> We frequently insert and remove elements from the beginning of a collection.

A linked list may be more appropriate.

Now consider:

> We need extremely fast average-case lookup using a key.

A hash table may be appropriate.

The important skill is therefore:

> **Match the data structure to the operations the application performs most frequently.**

2.3 Asymptotic Notation

Asymptotic notation allows us to describe how an algorithm behaves as the input size becomes large.

2.3.1 Growth Rates

Suppose we have three algorithms:

$T1(n)=10n$, $T2(n)=n^2$ and $T3(n)=2^n$

For small values of n, the differences may not appear significant.

For large n, however:

$n< n^2 <2^n$

This is why growth rate matters.

2.3.2 Big-O

Big-O provides an **upper asymptotic bound**.

If:

$T(n)=3n^2 + 5n +10$

then:

$T(n)=O(n^2)$

We ignore:

- constant factors
- lower-order terms

because the n^2 term dominates as n becomes large.

2.3.3 Big-Ω

Big-Ω provides a **lower asymptotic bound**.

If:

$T(n) = 3n^2 + 5^n + 10$

then:

$T(n)=\Omega(n^2)$

2.3.4 Big-Θ

Big-Θ provides a **tight asymptotic bound**.

For:

$T(n)=3n^2 + 5n + 10$

we can say:

$T(n)=\Theta(n^2)$

This tells us that the function grows asymptotically like n^2.

2.3.5 Common Data Structure Complexities

Students should eventually become comfortable with tables such as:

Operation	Array	Linked List	BST	Hash Table
Access	O(1)	O(n)	O(n)*	—
Search	O(n)	O(n)	O(n)*	O(1)*

Insert	O(n)	O(1)**	O(n)*	O(1)*
Delete	O(n)	O(1)**	O(n)*	O(1)*

* Depends on the structure and conditions.

** Assuming the relevant node/position is already known.

This table illustrates an important principle:

> **There is no universally best data structure.**

A structure that is excellent for one operation may be poor for another.

2.4 Time and Space Complexity

2.4.1 Basic Operation Counting

Consider:

```
int sum = 0;

for (int i = 0; i < n; i++)
{
   sum += i;
}
```

The loop executes n times.

Therefore:

$T(n)=O(n)$

Now consider:

```
for (int i = 0; i < n; i++)
{
   for (int j = 0; j < n; j++)
   {
      cout << i << j;
   }
```

}

The inner operation executes:

$n \times n = n^2$

times.

Therefore:

$T(n) = O(n^2)$

2.4.2 Input Size

Complexity is normally expressed in terms of the size of the input.

For an array:

n=number of elements

For a linked list:

n=number of nodes

For a tree:

n=number of nodes

For a graph:

we may use:

V=number of vertices

and:

E=number of edges

The appropriate measure of input size depends on the problem.

2.4.3 Best, Average, and Worst Case

Consider searching an array:

```
for (int i = 0; i < n; i++)
{
   if (data[i] == target)
      return i;
}
```

Best Case

The target is the first element.

O(1)

Worst Case

The target is the final element or does not exist.

O(n)

Average Case

The target is found somewhere in the middle on average.

Typically:

O(n)

2.4.4 Time Complexity

Time complexity describes how the amount of computational work grows with input size.

Examples:

Array access	O(1)
Linear search	O(n)
Binary search	O(log n)
Nested loops	$O(n^2)$

The goal is not necessarily to determine the exact number of CPU instructions.

Instead, we want to understand how the algorithm **scales**.

2.4.5 Space Complexity

Space complexity describes how much additional memory an algorithm requires.

For example:

```
int sum = 0;

for (...)
{
    ...
}
```

uses a constant amount of additional storage.

Therefore:

O(1)

If an algorithm creates an additional array of n elements:

O(n)

additional space is required.

2.4.6 Comparing Alternative Implementations

Consider implementing a Stack.

Array-based Stack

Stack

| A | B | C | D |

Advantages:

- simple
- fast access
- good cache locality
- no per-node pointer overhead

Potential limitation:

- capacity management

Linked-list Stack

Top
 |
 v
[A] -> [B] -> [C] -> NULL

Advantages:

- dynamic size
- no need for contiguous memory

Potential disadvantages:

- pointer overhead
- dynamic allocation
- poorer cache locality

Both implement the same Stack ADT.

The implementation choice depends on the requirements